Project Summary: 8-Channel ADC Peripheral for SCOMP

Atharva Kulkarni

Submitted
April 28, 2026

Introduction

This document covers the design and implementation of an SCOMP peripheral built around the LTC2308 8-channel, 12-bit analog-to-digital converter (ADC) on the DE10-Standard development board. It is written for engineers who may extend this work or use it as a reference when building their own SCOMP peripherals. The peripheral autonomously samples all eight analog input channels over SPI, making each result immediately available at I/O addresses 0xC0 through 0xC7. Our primary objective was to reduce the programmer interface to its simplest form: eliminating polling loops, wait states, and configuration registers. In sum, this peripheral satisfies every project requirement: it leaves the SCOMP processor unmodified, operates exclusively within the designated I/O address range, and presents an interface clean enough that reading an analog voltage only requires reading from the wanted channel.

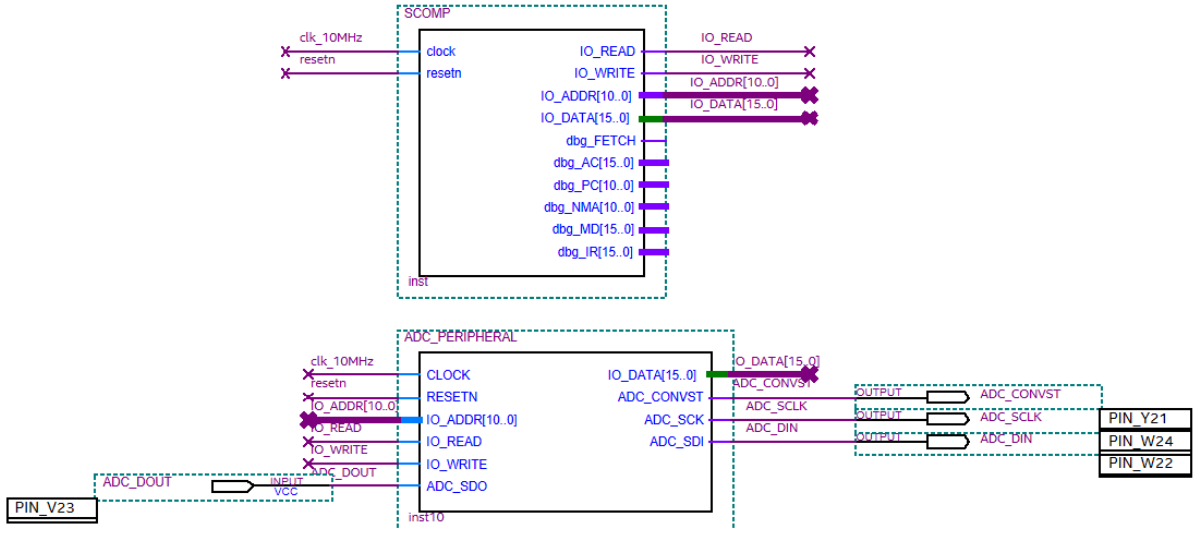

Figure 1. SCOMP-to-ADC peripheral interface and pin assignments.

Device Functionality

This peripheral requires no configuration from the SCOMP programmer. Upon reset, it automatically and continuously cycles through all eight ADC channels using the LTC2308's SPI interface, storing the most recent 12-bit conversion result for each channel in a dedicated register. Channels are sampled in sequence approximately every 25 microseconds per channel at a 10 MHz system clock.

To read a channel, the programmer issues a single IN instruction to the corresponding address. Successive IN instructions to different addresses return independent results, enabling simultaneous multi-channel reads. The returned value is a 12-bit unsigned integer in the range 0x000 to 0xFFF, representing a voltage between 0 V and 4.096 V in unipolar mode, with a resolution of 1 mV per LSB. No channel selection, no wait states, and no polling are required; results are always current. The peripheral uses LPM_BUSTRI to tri-state the IO_DATA bus when not addressed, allowing it to coexist safely with other peripherals.

ADDRESS	REGISTER NAME	ACCESS TYPE	B15	B14	B13	B12	B11	B10	B9	B8	B7	B6	B5	B4	B3	B2	B1	B0
0xC0	ADC_RESULT_CH0	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC1	ADC_RESULT_CH1	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC2	ADC_RESULT_CH2	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC3	ADC_RESULT_CH3	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC4	ADC_RESULT_CH4	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC5	ADC_RESULT_CH5	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC6	ADC_RESULT_CH6	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]
0xC7	ADC_RESULT_CH7	Read	-	-	-	-	B[11]	B[10]	B[9]	B[8]	B[7]	B[6]	B[5]	B[4]	B[3]	B[2]	B[1]	B[0]

Table 1. Peripheral Register Map with 12-bit result alignment

Design Decisions and Implementation

Our initial design followed a single-channel select model, where the SCOMP programmer would write a channel number to one address and read the result from another. Since this conflicted with our goal of a simple, configuration-free interface, we moved to a free-running design that samples all eight channels autonomously. This introduced a channel counter, per-channel result registers, and a background state machine, while reducing the programmer's interaction to a single IN instruction.

The LTC2308 controller in the starter project hardcoded its SPI configuration word to a single channel. We modified it to accept the configuration bits as an input port, allowing the peripheral to steer each conversion to the correct channel as it cycled through the sequence.

Finally, during hardware testing we discovered the physical ADC connector on the DE10 board is wired 180 degrees opposite from the schematic labels. We corrected this in VHDL using the expression (7 − ch_counter) when constructing the SPI configuration word, ensuring address 0xC0 maps correctly to physical pin ADC_IN0.

Conclusions

Therefore, this peripheral meets all requirements: all eight channels are readable via one IN instruction, SCOMP remains unmodified, and the design stays within reserved addresses. A live demo application confirmed reliable performance and an intuitive API.

Looking back, the single-channel select model should have been ruled out during planning rather than abandoned mid-development; the pivot cost time on both the VHDL and SPI controller work. If we revisited this project, we would commit to the free-running, all-channels architecture from day one and verify physical pin assignments against the board before finalizing any address decoding logic.

For engineers extending this work, the most valuable addition would be bipolar input mode via a writable UNI-bit control register, expanding the input range to ±2.048 V. A channel-freshness status register would also benefit latency-sensitive applications. The free-running approach trades sampling-time control for interface simplicity, a trade-off worth considering in timing-critical systems. Overall, this peripheral is a reliable foundation for SCOMP analog sensing.